

Richer State Models

The state model is the type lattice — and it was built two phases ago

2026-08-29

The one-sentence thesis

The system-model tier proves behavioural properties of a process by modelling its state as **one integer**. Everything you'd want next — multiple variables, records, collections, sum-typed protocols, and invariants **discovered rather than annotated** — is not a second research programme. It is a **wiring** job between two things that already exist and were built to fit: the **tag lattice** from the typing phase (which already infers the shape of every value) and the **guarantee engine** from the system phase (which already proves over SMT). This document shows the seam, proves the two load-bearing claims with runnable spikes, and lists what to build.

THE THROUGH-LINE

The typing phase inferred **shapes**. The system phase proves **behaviour over shapes**. “Richer state models” is the missing wire between them — **type-directed SMT translation**. The information is not missing; it is inferred and sitting one namespace away.

What “state” is today, exactly

Read `priv/system/core.bl`. `verify-process` collects the state variables an invariant mentions, and then:

```
svars (state-vars inv-node clauses)
svar  (if (empty? svars) nil (first (into [] svars))) ; ← takes the FIRST
```

And `priv/system/smt.bl` states its own ceiling honestly:

“State vars are modelled as SMT integers — the common case for process state (balances, counters, phases-as-ints). Non-integer state (maps, strings) is out of this translator’s scope and surfaces as `:untranslatable`.”

So the model of a process’s state is: **one integer that moves**. `balance =`, guarded by `5`

integer arithmetic, discharged by `z3`. When state is anything else, the translator returns `:untranslatable` and the checker stays silent rather than lie. That is a real and sound floor — and it is a floor, not a ceiling, only because the richer information is being thrown away, not because it is unavailable.

Claim 1 — most of the work is already done

The lift to richer state has two halves: **infer the structure** and **prove over it**. The first half is the entire typing phase, already shipped. The second half is the guarantee engine, already shipped. What is missing is the adaptor between them. Here is the inventory.

capability	already built, where	status
value shapes	<code>typed/all-tags</code> lattice: <code>:int :float :map :vec :set :string</code> — <code>:bool :fn :nil :sym :seq :list</code> every value's tag inferred by <code>typed/walk</code>	✓ shipped
flow narrowing	<code>walk-if</code> / <code>guard-narrow</code> : <code>(if (int? x) ...)</code> re-fines <code>x</code> to <code>:int</code> in the then-branch, <code>:diff</code> in the else — path-sensitive	✓ shipped
record fields	<code>bind-pattern</code> deconstructs <code>{:keys [...]}</code> and map patterns, binding each field	✓ shipped
signatures from source	<code>sigs-from-env</code> reads <code>^{:args :ret}</code> off name meta through the real compile pipe — no re-declaration	✓ shipped
multi-var SMT	<code>preserves?</code> already declares <code>svar</code> + <code>svar2</code> + inputs and primes; N was a loop where there was a <code>first</code> — now generalized	✓ shipped
candidate predicates	<code>system.core/candidates</code> — miniKanren enumerates <code>(op a b)</code> over a domain, deduped	✓ shipped
z3 preservation	<code>preserves?</code> / <code>sufficient?</code> / <code>establishes?</code> — the Hoare triple discharge	✓ shipped
effect / footprint	<code>system.footprint/fp-walk</code> reads resource modes from a body — never annotated	✓ shipped

Every row is now **shipped**. The last to land, Rung 1, was genuinely a truncation bug: `state-vars` collected the set of state variables, and `verify-process` kept one. The z3 layer beneath it (`preserves?`) already handled a current var, a primed var, and declared inputs — it was always N-ary; only the seam narrowed it to one. Fixing that opened the whole ladder.

Proof: multi-variable relational invariants (Rung 1)

To show the machinery generalizes, `research/pl8_state_models/spike_multivar.bl` models a two-variable state with a **relational** invariant — one that a single-integer model **cannot** express:

```
state      {:balance b, :reserved r}
invariant  (and (>= reserved 0) (>= balance reserved))
           ;; reserved funds are non-negative AND never exceed the balance
```

 Clojure

The check declares the whole tuple, primes it jointly (`balance2`, `reserved2`), and asks z3 whether any transition can break the coupled predicate.

```
$ elixir ... research/p18_state_models/spike_multivar.bl
```

```
reserve preserves invariant: true
release preserves invariant: true
withdraw preserves invariant: true (guards against FREE funds)
buggy withdraw preserves invariant: false (guards whole balance)
PASS: three good transitions proven, the reservation-ignoring bug caught
```

The relational invariant `balance ≥ reserved` is proven for three transitions, and a buggy `withdraw` that guards the whole balance (ignoring the reservation) is caught with the exact reasoning a single-variable model is blind to: it can drive `balance < reserved`. The z3 mechanism needed **no change** — only the seam that today writes `(first svars)` needs to declare and prime the whole set.

Claim 2 — the way it was built leads to crazier things

Here is the part worth slowing down for. The typing phase did not build a type **checker**. It built a **relation** and a set of **engines**, and a relation runs in every direction. That design choice — visible in `typeo` and in the strict engine division — means the richer-state ladder does not just climb to “more types”. It climbs to capabilities that were never on the original list.

The relation runs backwards: synthesis, not just checking

`examples/typing/04_holes_demo.bl` defines `typeo`, a bidirectional typing relation in miniKanren. The demo’s own words:

“The relation is the same artifact that checks (forwards), refutes (sideways), and synthesizes (backwards): `typeo`.”

A checker answers “does this term have this type?”. A **relation** also answers “**what terms have this type?**” — run `typeo` with the term as the unknown and it **enumerates inhabitants**. The holes demo fills a typed hole with real expressions from the local context; the ill-typed hole gets no answer.

Now apply that to **state**. Once state has a type richer than `Int`, “what transition preserves this invariant?” is the **same backward run**. The tier stops being “prove the invariant the human wrote” and becomes “**synthesize the guard that makes the invariant hold**” — program repair, from the same relation, for free.

The engines were kept strictly non-overlapping — so they compose

The system phase inherited a discipline from the typing phase: **four engines, no fallbacks, strict ownership**.

tag lattice owns structure · **z3** owns arithmetic decisions · **miniKanren** owns relations and synthesis · **datalog** owns the codebase-as-facts. Effects are a fifth, folded in as a footprint lattice.

Because — each engine answers a different KIND of question, so a richer state model recruits all four without any one leaking into another’s territory

That non-overlap is why the ladder is composable rather than a rewrite. A sum-typed protocol state, at the summit, uses **all four at once**: the lattice gives the variant tags, z3 proves the per-

variant arithmetic, miniKanren synthesizes the missing transition, datalog checks every **caller** speaks the protocol. None of them has to learn another’s job.

The crazier thing, proven: invariants discovered from nothing

The boldest consequence. Today you must write `^{:invariant (>= balance . But the 0)}`

machinery to **discover** it is already present: `candidates` enumerates predicates, `preserves?` decides them, and the state vars come from the lattice. Assemble those into a **Houdini fixpoint** (Flanagan–Leino) — start from every candidate that holds at the initial state, drop any conjunct a transition can break, iterate — and the invariant **falls out of the code**.

`research/p18_state_models/spike_discover.bl` runs exactly this on an **unannotated** account machine:

```
$ elixir ... research/p18_state_models/spike_discover.bl

candidate vocabulary (6): (<= balance 0) (<= balance 1) (<= balance 5)
                        (>= balance 0) (>= balance 1) (>= balance 5)
hold at init (balance=0): (<= balance 0) (<= balance 1) (<= balance 5) (>= balance 0)
^ survive induction:      (>= balance 0)
DISCOVERED INVARIANT: (and (>= balance 0))
PASS: discovered (>= balance 0) from nothing; refuted the bogus candidates
```

From an account with **no annotation**, the loop discovers `(>= balance 0)` and refutes every bogus candidate (`<= balance 0`, `>= balance 5`, ...). The one subtlety, pinned in the spike: Houdini must be seeded with a **consistent** set, so the establishment filter (what holds at `init`) runs **before** induction — otherwise the contradictory candidates make the antecedent unsatisfiable and induction is vacuous. Order matters; the result is real.

This is **abduce**’s existing search aimed at the **whole** invariant instead of a missing conjunct, seeded by the inferred state vars. “Implied invariants” was never a missing engine — it was a loop we had not yet written around engines we already had.

The ladder

Each rung is a concrete capability, and each names the existing machinery it recruits. The difficulty is honest: the top two rungs cross z3’s decidability line and pay for it in the **approximate-guarantee catalog** the system tier already ships (`exact-guarantees` vs `approximate-guarantees`).

rung	state model → what it buys	warm
1 · multi-var	tuple of ints; relational invariants (<code>balance ≥ reserved</code>). Rate limiters, pools, coupled counters. Shipped — demo 10.	21 ms
2 · records	product of typed fields (<code>int × bool</code>); mode/flag machines (<code>frozen ⇒ balance = 0</code>). Each field its own z3 sort, read from init. Shipped — demo 11.	23 ms

rung	state model → what it buys	warm
3 · collections	capacity via length abstraction (<code>count ≤ 10</code>), EXACT; content (no-duplicate) is the bounded companion. The catalog keeps them apart. Shipped — demo 12.	19 ms
4 · sum types	tagged variant as a phase (<code>:idle :open :closed</code>); session-type conformance (<code>send ⇒ open</code>), use-after-close caught. Shipped — demo 13.	17 ms
★ · discovery	invariant synthesis via Houdini over the state vars. No annotation. Shipped — demo 14.	99 ms

TYPE-DIRECTED TRANSLATION IS THE WHOLE MECHANISM

Today `smt.bl` hardcodes `Int`. The lift is: for each state var, ask the tag lattice its type and emit the matching z3 sort — `Int`, `Bool`, `(Seq ...)`, `(Array ...)`, a declared datatype. `state ≜ [[type-lattice]]`. The translator stops assuming and starts **consulting**. That single indirection is Rungs 2–4.

What is already inferred (the “implied everything” audit)

The user asked specifically what is **implied** rather than annotated. The answer is: nearly everything except the top-level invariant, and even that is now in reach.

property	inferred?	where
types	✓	the entire tag-lattice phase — no annotation needed
effects / footprints	✓	<code>fp-walk</code> reads resource modes from the body
process name (when unnamed)	✓	<code>model/content-hash</code> — <code>υ(labels) · hash</code> over the transition graph
handled message protocol	✓	<code>handled-labels</code> reads the accepted set from the clauses
unhandled messages	✓	join of <code>:call/*</code> senders against handled — the check TLA^+ cannot make
missing preconditions	✓	<code>abduce</code> — miniKanren enumerates, z3 decides, shortest-first
return types	✓	<code>codebase/infer-clause-ret</code> — <code>typed/walk</code> over the body when unannotated
termination measures	●	<code>termination</code> proves <code>dec / rest</code> shrinking; <code>^{:decreasing}</code> is the escape hatch

property	inferred?	where
the top-level invariant	★	was annotate-only; the discovery spike above closes it

abduce **strengthens** a given invariant with a missing conjunct; **all-senders-guarantee?** **infers a whole-program obligation** from the call graph. Discovery is those same engines run as a fixpoint over the lattice’s state vars — the last “annotate-only” property becomes inferred.
Because — the two auto-strengthening pieces already ship, and discovery is the same search with a wider target

The demos it produced

The work was filed as **PLAN-050** — six pieces, dependency-ordered, each shipping as **tests** and a **demo** (the tier’s graduation contract) — and all six are done. The demos are the proof surface; every one runs clean:

demo

examples/system/10_multivar_invariant.bl

examples/system/11_record_state.bl

examples/system/12_collection_state.bl

examples/system/13_protocol_state.bl

examples/system/14_discover_invariant.bl

examples/system/15_type_directed.bl

the story it tells

a reservation account:
balance ≥ reserved ≥ 0 proven;
the reservation-ignoring withdraw caught — a bug invisible to single-variable models

a mode machine:
frozen ⇒ balance = 0 over
a **{:balance int :frozen bool}**
record; a transition that pays out while frozen caught — mixed-sort state

a bounded work queue: “the pending queue never exceeds capacity” as a **bounded** guarantee, registered honestly in the approximate catalog

a connection with **:idle | :open | ; :closed**
an illegal **:send**-while-**:closed** transition caught — sum-typed protocol conformance

point the checker at an **un-annotated** server; it **prints the invariant it discovered** and then proves it — the headline

the same server, three state shapes (int, record, sum); the **one** translator adapts

demo

the story it tells

by consulting the lattice — showing the mechanism, not just the results

Where it stands

The whole ladder is **shipped**, not spiked. One `verify-process` proves an invariant over five state shapes — a scalar int, a tuple of ints with a relational invariant, a mixed-sort record (int $\times$ bool), a phase-coded variant (protocol conformance), and a collection abstracted to its length — each by reading the field sorts from the init state, each with its bug caught and rendered like an ordinary compiler warning. And the invariant itself is **discovered from nothing** by a Houdini fixpoint when there is no annotation. `examples/system/10 – 15` demonstrate every rung; the seam tests grew from 9 to 21 cases; the non-auth suite is green at **647 tests / 1760 assertions / 0 failures**.

Two bugs were fixed at the root along the way, both found by a rung disagreeing with an obviously-correct case: `errors/delaborate` had no map case (record state rendered as a raw tuple), and `free-syms` declared the boolean literals `true / false` as integer inputs, shadowing SMT's booleans and silently breaking any transition that assigned a bool literal. Neither was worked around.

Nothing here is a new engine. All of it is the wire between two phases that were, in retrospect, built to be connected: the type lattice infers the shape, the guarantee engine proves over it, and “richer state models” was the type-directed translation between them. That is the sense in which most of the work was already done — and the sense in which the way it was done reached further than the original proposal: a relation that synthesizes, four engines that compose, and an invariant that no longer needs a human to state it. The remaining depth is genuine (z3-sequence CONTENT invariants beyond length; a richer candidate vocabulary for discovery) but it is depth on a shipped foundation, not a missing floor.